

Thread 3 — Setting up Lean

AI Agents for Mathematics Research · setup and troubleshooting

Department of Mathematics, University of York

On a classroom machine

Lean, VS Code and the module repository are pre-installed and the Mathlib build cache is pre-warmed. You need only:

```
cd ~/thread3
git pull
code .
```

If `~/thread3` does not exist, or `git pull` fails, tell a demonstrator. Do not spend the session fixing it yourself; the image is our problem, not yours.

In a browser, with nothing installed

You do not have to install anything to do this module. The repository carries a **dev container**: a description of the environment, which GitHub can build and run for you in the cloud. You get the same VS Code, the same Lean, the same pinned Mathlib and the same Infoview as everyone else, in a browser tab.

On the repository page on GitHub, click the green **Code** button, then the **Codespaces** tab, then **Create codespace on main**. Wait. The first time takes a minute or two; after that, reopening an existing codespace is quick.

When the editor appears, the terminal at the bottom will finish with `Setup finished`. Open `Thread3/Week01.lean` and wait for the Infoview to appear on the right. That is the whole setup.

Two things to know, and they are the only two:

- **Your codespace stops after 30 minutes of inactivity.** Your files are kept — reopen it from the same Codespaces tab and carry on. It is stopping, not being deleted.
- **Commit and push your work at the end of every session.** A codespace is not a backup. It is also how a demonstrator can look at your file when you ask for help outside the session.

```
git add -A
git commit -m "week 1"
git push
```

There is a monthly allowance of cloud hours attached to your account. The thread is well inside it, but do not leave a codespace idling for days, and delete old codespaces you have finished with (same Codespaces tab, `...` ► **Delete**) — a stored codespace uses allowance even when stopped. If you sign up for the **GitHub Student Developer Pack**, which is free and takes five minutes, the allowance goes up by half again.

For this module the browser environment is a genuine equal of a local install — use it all term if you prefer. Work locally if you would rather not depend on the network during a practical, or if you already have Lean set up.

On your own machine

You need about **10 GB of free disk** and a working `git`.

1. Install `elan` (the Lean version manager). It reads the repository's `lean-toolchain` file and installs exactly the right Lean version, so never install Lean any other way.

```
curl https://elan.lean-lang.org/elan-init.sh -sSf | sh -s -- -y
```

On Windows, use the installer from the Lean website rather than the shell script, or work inside WSL.

2. Install VS Code and, from the Extensions panel, the extension called `Lean4`, publisher `leanprover`. Do not install any other Lean extension.

3. Clone the repository and fetch the build cache.

```
git clone <module repo URL> thread3
cd thread3
lake exe cache get
```

`lake exe cache get` downloads Mathlib pre-built. It takes a few minutes and several gigabytes. **Do not skip it** — without it, Lean will build Mathlib from source, which takes hours.

4. Open the folder in VS Code — `code .` from inside `thread3`, or File ► Open Folder. Opening a single `.lean` file instead of the folder is the single commonest setup mistake: Lean then cannot find the project and nothing works.

5. Check it. Open `Thread3/Week01.lean`. Within a minute or so the Infoview should appear on the right showing goals. If it does, you are done.

Using it

Infoview	opens automatically; reopen with Ctrl/Cmd+Shift+Enter
Goals	shown in the Infoview for wherever your cursor is
Errors	red squiggles, and the Problems panel (Ctrl/Cmd+Shift+M)
Unicode	type <code>\to</code> , <code>\forall</code> , <code>\R</code> , <code>\N</code> , <code>\le</code> , <code>\in</code> , <code>\sum</code> , <code>\<></code> and press space or tab
Restart a file	Ctrl/Cmd+Shift+P ► “Lean 4: Restart File”

A `sorry` produces a **warning**, not an error, and the file still builds. That is why “it builds” is not the standard for finished work — no `sorry` and an empty Problems panel is.

Troubleshooting

The Infoview never appears, or says “Waiting for Lean server”. On your own machine, you opened a file rather than the folder: close everything, open the `thread3` folder, and try again. In a browser codespace the folder is always open, so this instead means the Lean server is still starting — give it a minute, then **Ctrl/Cmd+Shift+P** ► “Lean 4: Restart File”.

Everything is underlined red, including `import`. Mathlib is not built. Run `lake exe cache get` in the project root, then restart VS Code.

An `import` you added yourself is underlined red, but the rest of the file is fine. In a codespace, Mathlib comes pre-built for the imports listed in `Thread3/Common.lean` — which is everything the worksheets need — rather than for all eight thousand of Mathlib’s files. If you reach further into the library, fetch the rest once:

```
lake exe cache get
```

That takes a few minutes, once, and then any import works. If instead it starts building Mathlib from source, stop it and see the next entry.

It is building Mathlib from source (thousands of files scrolling past). Stop it. `lake exe cache get` failed — usually a network problem or a mismatched Lean version. Check that `elan show` reports the version in `lean-toolchain`.

lake: command not found. `elan` is not on your `PATH`. Open a new terminal; if that does not fix it, on macOS or Linux add `source $HOME/.elan/env` to your shell profile.

A lemma name that works for a classmate fails for you. You are on a different Mathlib revision. Run `git pull` and `lake exe cache get`. Mathlib renames things, and the module pins one revision for exactly this reason — never edit `lean-toolchain` or `lakefile.toml`.

Everything worked yesterday and now nothing does. Try “Lean 4: Restart File”, then restart VS Code, then `lake exe cache get` again, in that order.

It is very slow. Lean recompiles the whole file on every keystroke-idle. Large `simp` calls, `exact?` and `decide` are genuinely slow; that is normal. If the whole file is slow, check you have the cache and not a source build.

In a codespace: my work has vanished. You almost certainly created a *second* codespace; each one has its own copy of the files. Go to the repository’s Code ► Codespaces tab, reopen the older one, and push from it. This is why the instruction is to commit and push at the end of every session.

In a codespace: the tab says the codespace is stopped or disconnected. Normal after 30 minutes of inactivity. Reload the page, or reopen it from the Codespaces tab. Nothing is lost.

In a codespace: it says “no valid machine types available”. A repository configuration problem, not something you can fix. Tell the module lead.

Getting help

In the session, ask a demonstrator — that is what they are there for, and being stuck for ten minutes on a setup problem is a waste of the fifty. Outside the session, post the **exact error text** in the module forum, not a screenshot and not a description. Include the output of `elan show` and the first line of `git log --oneline -1`. Anyone can then reproduce your problem in a minute.